arXiv:2601.04505v3 [cs.AI] 26 May 2026

CircuitLM: A Multi-Agent LLM-Aided Design Framework for Generating Circuit Schematics from Natural Language Prompts

Khandakar Shakib Al Hasan[1], Syed Rifat Raiyan[1], Hasin Mahtab Alvee[1], Wahid Sadik[2]
[1]*Department of Computer Science and Engineering*, [2]*Department of Electrical and Electronic Engineering*
Islamic University of Technology
Dhaka, Bangladesh
{shakibalhasan, rifatraiyan, hasinmahtab, wahidsadik}@iut-dhaka.edu

***Abstract*—Generating accurate circuit schematics from high-level natural language descriptions remains a persistent challenge in electronic design automation (EDA), as large language models (LLMs) frequently hallucinate components, violate strict physical constraints, and produce non-machine-readable outputs. To address this, we present CircuitLM, a multi-agent pipeline that translates user prompts into structured, visually interpretable `CircuitJSON` schematics. The framework mitigates hallucination and ensures physical viability by grounding generation in a curated, embedding-powered component knowledge base through five sequential stages: (i) component identification, (ii) canonical pinout retrieval, (iii) chain-of-thought reasoning, (iv) JSON schematic synthesis, and (v) interactive force-directed visualization. We evaluate the system on a dataset of 100 unique circuit-design prompts using five state-of-the-art LLMs. To systematically assess performance, we deploy a rigorous dual-layered evaluation methodology: a deterministic Electrical Rule Checking (ERC) engine categorizes topological faults by strict severity (Critical, Major, Minor, Warning), while an LLM-as-a-judge meta-evaluator identifies complex, context-aware design flaws that bypass standard rule-based checkers. Ultimately, this work demonstrates how targeted retrieval combined with deterministic and semantic verification can bridge natural language to structurally viable, schematic-ready hardware and safe circuit prototyping. Our code and data will be made public.**

***Index Terms*—Circuit Design, CircuitJSON, CircuitLM, Electronics Design Automation (EDA), Embedded Systems, Firmware Generation, Chain-of-Thought Reasoning, Hardware Synthesis, Large Language Models (LLMs), Multi-Agent Systems, Retrieval-Augmented Generation (RAG), Schematic Generation, Vector Database**

I. Introduction

Electronics design requires deep expertise, precise component knowledge, and manual schematic drafting—barriers that exclude novices and rapid prototypers. Recent LLMs promise natural-language circuit design (*e.g.*, "Build a gesture-control circuit with an ESP32-C3 Mini and five MPU6050 sensors"), yet in practice, they generate unstructured textual wiring descriptions that are ambiguous, incomplete, and electrically unsound. Connections are often inaccurate, essential supporting components (pull-ups, decoupling capacitors, current-limiting resistors, I^2C multiplexers) are omitted, and power distribution constraints are ignored.

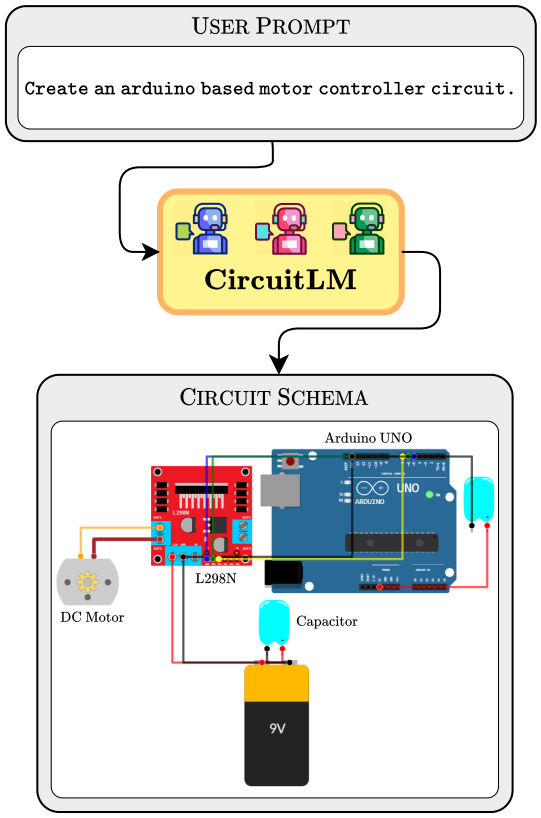


Fig. 1. Example of a circuit schema generated by CircuitLM from one of our benchmark prompts.

Crucially, these prose outputs are incompatible with standard EDA tools such as *Fritzing*[1] and *Proteus Design Suite*[2]. Users must manually reconstruct the design in an EDA environment—a time-consuming process that frequently exposes additional logical and electrical faults. Consequently, LLM-generated circuits are rarely functional or safe on first pass.

Existing benchmarks (PINS100, MICRO25) [1] lack (1)

[1]https://fritzing.org/
[2]https://www.labcenter.com/

holistic electrical validity checks, (2) structured machine-readable outputs, and (3) automated visualization.

To address this limitation, we introduce **CircuitLM**, a multi-agent framework that converts natural-language prompts into reliable, executable circuit schematics. Grounded in a local vector database of canonical production components, CircuitLM enforces strict pin canonicalization and invariant wiring constraints (*e.g.*, SDA↔SDA consistency, mandatory current limiting) through a five-stage pipeline. Beyond standalone design, CircuitLM can act as a circuit design assistant if paired with platforms such as Tinkercad or Wokwi. The system can accelerate prototyping and lower the barrier for users to create and iterate on circuits, reducing the time from idea to a testable design.

We evaluate CircuitLM on a curated dataset of 100 prompts against five frontier LLMs using a rigorous ERC framework, backed by expert human validation. Our methodology evaluates circuits by:

- **Categorized Severity**: Deterministically classifying topological and logical faults into Critical, Major, Minor, and Warning tiers, mirroring industry-standard EDA tools.
- **Pass@1 Success Rates**: Establishing absolute ground truth via human experts to define a strict success criterion (0 Critical or Major errors) for prototyping-ready schematics.

Our contributions are fourfold. First, we present CircuitLM, a multi-agent pipeline for automated circuit generation and interactive visualization from natural language. Second, we introduce `CircuitJSON`, an open, tool-agnostic schematic interchange format for structured hardware representation. Third, we establish a deterministic ERC evaluation framework with a 100-prompt benchmark dataset to assess the physical validity of LLM-generated circuits. Finally, we develop a retrieval-augmented architecture that eliminates component hallucination. By decoupling the prompt context from the total database size (N), the framework scales to industrial libraries with $\mathcal{O}(k)$ prompt complexity, where k is the number of components required for a given design.

II. Related Works

Electronic circuit design automation has evolved from deterministic, heuristic-driven algorithms toward stochastic, agentic AI frameworks, reflecting the increasing complexity of embedded systems and the demand for higher-level abstraction in hardware synthesis. Early applications of Machine Learning (ML) in Electronic Design Automation (EDA) focused on localized optimization within the physical design flow, employing Convolutional Neural Networks (CNNs) for routability prediction and Graph Neural Networks (GNNs) for netlist representation and parasitic estimation [2]. While effective at specific VLSI stages, these approaches lacked the semantic capacity to interpret high-level user intent or natural language specifications. The success of LLMs in software engineering subsequently motivated research into hardware generation, particularly in Hardware Description Languages (HDLs). Systems such as VerilogEval [3] and ChipChat [4] demonstrated LLM-based RTL generation via conversational interfaces. However, generating complete electronic schematics introduces substantially higher complexity, requiring physical grounding through accurate pin mappings, power management, and integration of heterogeneous peripherals. A key challenge in autonomous schematic generation is the hallucination of connectivity, where models invent invalid pin labels or violate electrical constraints. Recent work has explored structured intermediate representations to mitigate this issue. Schemato [5] reconstructs schematics from formal netlists, while EESchematic [6] investigates end-to-end generation using standardized symbol libraries. Nonetheless, existing datasets and frameworks—such as LLM4Netlist [7]—remain focused on synthesizable digital logic, leaving embedded systems and analog sensor-rich designs largely unaddressed. To mimic human engineering workflows, research has pivoted toward Multi-Agent Systems (MAS), which have already demonstrated good efficacy in mathematical [8] and physics [9] reasoning. By decomposing design into specialized roles—such as design, verification, and routing agents—MAS frameworks employ "debate and critique" mechanisms to reduce error rates [10]. Cutting-edge frameworks like MenTeR [11] utilize diagram-aware RAG (Retrieval-Augmented Generation) and "circuit think tanks" for automated RF/analog design. Furthermore, studies on agent topologies suggest that distributed reasoning consistently outperforms monolithic models by isolating sub-task complexity [12].

As evident in Table VI, CircuitLM bridges these gaps by introducing a novel multi-agentic pipeline that enforces physical grounding through a canonical retrieval database. By moving beyond simple text generation to a structured, reasoning-first approach, this study provides the first comprehensive evidence that agentic frameworks can handle the multidimensional constraints of real-world embedded prototyping, producing rapid-prototyping outputs that are both logically sound and visually interpretable.

III. Methodology

We propose a multi-step, multi-agent pipeline that transforms a high-level, natural language project idea (the user prompt) into a structured, logically sound circuit definition. The pipeline contains the following 5 stages:

A. Stage I: Components Extraction (Identification Agent)

In the initial phase, a specialized **Identification Agent** parses the user prompt and performs named entity recognition (NER) plus intent analysis to extract a comprehensive hardware list. To constrain the search space and reduce tokens, the agent is limited to generic component categories (*e.g.*, Pressure Sensor, Arduino Uno, IMU Sensor) rather than full component records. The output is a structured JSON array of component names (*e.g.*, `["Microcontroller", "Motor Driver", "DC motor"]`). Figure 3 shows the identification agent returning the required component list.

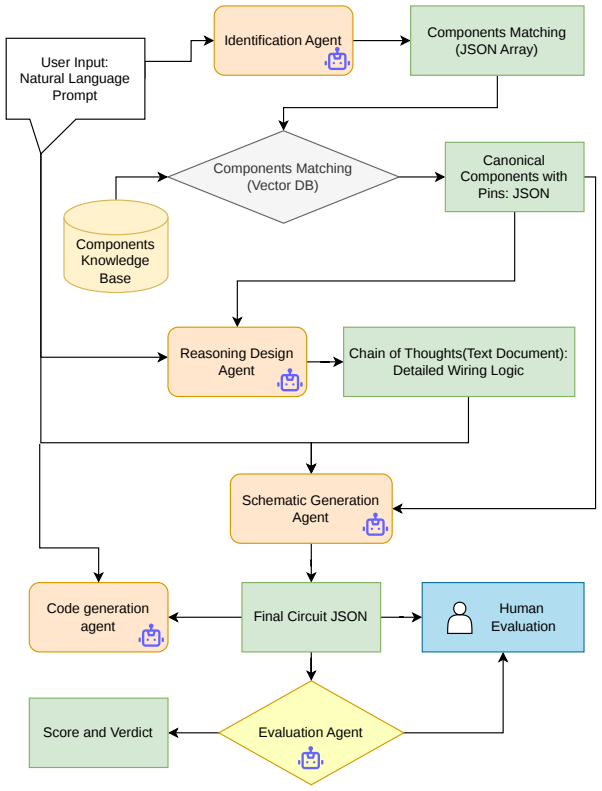


Fig. 2. An overview of the CircuitLM framework.

B. Stage II: Component Matching (Retrieval Agent)

The list of Stage I is passed to the **Retrieval Agent**. The agent runs an embedding-based similarity search over a local vector knowledge base of canonical components [13], then augments it with a fuzzy lookup using a dictionary of equivalent parts. Results are merged and mapped to records in the `component` database, yielding a dictionary of verified canonical names and precise pin-out definitions (Figure 4). This step prevents LLM pin hallucinations by sourcing exact pin names and numbers from the curated database. If a requested part is missing, an Out-of-Distribution (OOD) flag is raised, execution halts, and human review is requested. OOD halt is a strict safety guardrail to prevent hallucinated wiring.

C. Stage III: Design Reasoning (Electronics Expert Agent)

The **Electronics Expert Agent** is the cognitive core of the pipeline. Given the user prompt, validated component list, and pin definitions, it performs high-level engineering reasoning and produces a structured Chain-of-Thought (CoT) document [14]. The CoT explicitly encodes functional goals, power requirements, safety constraints, and pin-level wiring logic. Using least-to-most prompting [15], the agent decomposes complex requirements into hierarchical sub-tasks, improving electrical logicality prior to code generation [16]. Figure 6 shows the CoT agent inputs and outputs.

D. Stage IV: Schema Generation (Circuit Generation Agent)

The **Circuit Generation Agent** transforms the CoT document and canonicalized pin mappings into a machine-readable schematic representation (Figure 5). It produces a strictly formatted `CircuitJSON` object, inspired by the Wokwi[3] simulation platform [17]. The output specifies exact component placement (coordinates and rotation) together with a complete, canonicalized list of pin-to-pin electrical connections. Each component entry additionally contains a dedicated `attrs` field for parametric configuration. This field stores structured key–value pairs representing electrical parameters (*e.g.*, resistance, capacitance, inductance), operating constraints (*e.g.*, voltage limits), or optional metadata required for simulation and validation. By separating structural connectivity from parametric specification, the schema ensures deterministic wiring while preserving flexibility for component-level configuration. The resulting JSON is directly consumable by the schematic viewer; Listing 1 details the full structure.

E. Stage V: Schematic Visualizer

The browser-based visualization engine renders `CircuitJSON` into interactive SVG schematics, mapping recognized components to predefined symbols and unknown items to labeled rectangular glyphs. Initial component placement utilizes a force-directed algorithm [18] to group connected parts and prevent overlap, while wire routing applies a Manhattan-style strategy [19] prioritizing short paths and minimal bends. Crucially, this serves as a dynamic, human-in-the-loop interface allowing users to drag components and reshape wires in real-time. (Figure 5 and Figure 7 display the raw, unadjusted autonomous outputs, while the end-to-end pipeline is summarized in Figure 2).

Diverging from traditional, netlist-heavy EDA workflows, CircuitLM's visual-first paradigm is lightweight and optimized for early prototyping and education. Because the interactive interface allows seamless manual layout resolution, strictly optimizing deterministic graph-drawing metrics (*e.g.*, minimizing edge crossings) is unnecessary. Such routing optimizations remain a separate challenge outside our primary scope of evaluating LLM electrical reasoning.

F. Firmware Code Generation

While the primary pipeline focuses on circuit generation and evaluation, we include a firmware code-generation agent to demonstrate automated deployment (not part of the core workflow). For ESP32 and Arduino targets, we pass the prompt and generated `CircuitJSON` to the code agent. LLMs already perform strongly on software benchmarks (HumanEval [20], MBPP [21]); since embedded firmware often follows repeatable patterns, producing syntactically correct firmware is comparatively easier than hardware synthesis.

[3] https://wokwi.com/

G. ERC Evaluation Framework

While circuit simulation and ERC are standard in traditional EDA workflows, legacy tools are often poorly matched for LLM-driven, board-level generation. SPICE, the gold standard for continuous-time analog analysis, is computationally infeasible and practically irrelevant for our microcontroller-centric targets (*e.g.*, Arduino/ESP32), which are dominated by digital I/O and discrete firmware states. Similarly, traditional ERC engines rely on rigid, proprietary netlist formats unsuited for lightweight generative workflows. Rather than bypassing verification, we developed a custom, deterministic ERC engine engineered specifically to parse `CircuitJSON`. By modeling the generated schematic as a topological graph (via `networkx`), our framework performs programmatic pathfinding to verify physical safety and logical constraints. The engine evaluates circuits against a curated knowledge base, deterministically categorizing faults by severity (Critical, Major, Minor, Warning). The registry of automated checkers includes galvanic short-circuit detection, passive component verification (*e.g.*, current-limiting resistors for LEDs, pull-up resistors), inductive load protection (flyback diodes), logic-level voltage matching, and floating input detection. This tailored, graph-based approach allows us to rigorously evaluate the physical validity of LLM-generated schematics without the overhead of continuous-time analog simulation.

IV. Experimental Setup

The experiment was designed to ensure fair and robust benchmarking of the candidate LLMs. We run the experiments multiple times, and the final score is calculated as the mean across all runs. This setup contains the implementation of the knowledge base and the configuration of the LLM agents.

A. Local Knowledge Base

A key element of the system is the curated component library, stored in a ChromaDB vector database. This database serves two primary functions:

- **Components List**: Providing a finite and authoritative set of components, including their canonical identifiers, mandatory pin labels, and known aliases. This enables strict evaluation while still supporting flexible component retrieval.
- **Semantic Component Resolution**: Translating verbally requested components generated by the LLM in Stage I into definitive canonical component keys and pin sets required for the final `CircuitJSON` representation in Stage IV.
- **Database**: We choose ChromaDB for its lightweight architecture and schema flexibility, allowing rich component metadata (pins, aliases, specifications) to be stored alongside embeddings, simplifying the mapping from semantic queries to canonical component definitions. For this experiment, we utilize a set of 70+ components representing a broad range of component types. This selection establishes a foundational taxonomy of standard embedded interfaces (*e.g.*, I^2C, SPI, PWM). Because devices within the same family share identical wiring logic, this curated set comprehensively covers practical design patterns. Our primary objective is to demonstrate the framework's structural reasoning across distinct protocols; expanding the component library is simply a data-entry task.
- **Embedding Model**: `Qwen3-Embedding-0.6B` [22] model is used to compute dense vector representations of component descriptions. Component matching is performed using cosine similarity with an empirically determined similarity threshold. The Qwen3 embedding model provides high-quality semantic representations for short, technical phrases and structured component metadata in electronics domains. The model offers a favorable balance between retrieval accuracy and computational efficiency [23], making it well-suited for persistent vector search [13].
- **Library Structure**: Each component entry includes a canonical name (key), mandatory pin labels, physical dimensions (width and height), aliases, a natural language description, category, typical usage, and technical specifications. Strict adherence to the defined pin labels (*e.g.,* `D21`, `GND`, `VCC`) is enforced during final evaluation (Dimension 1).

B. Benchmarked LLMs

A total of five distinct LLMs were benchmarked as candidate models across the key text-generation stages of the proposed multi-agent pipeline (Stages I, III, and IV). The evaluated models are: `gpt-5-mini` [24], `gemini-2.5-flash` [25], `deepseek-v3.1` [26], `qwen3-235b-a22b-2507` [22], and `llama-3.3-70b-instruct` [27].

All models were accessed through a unified API interface to ensure consistency in prompt formatting and execution conditions, and the temperature parameter was fixed at 0 for all experimental runs.[4] For each experimental run, the same base model was used across all agents, ensuring that model performance was evaluated consistently without mixing different models across pipeline stages.

C. Dataset (User Prompts)

A prompt-based dataset for netlist generation was introduced in 2025 [7]. It focuses exclusively on synthesizable digital logic gate circuits in Verilog, excluding embedded systems, IoT, and Arduino-specific components such as PWM motor drivers and sensor interfaces. In contrast, our experiment utilizes a set of 100 diverse user prompts covering a broad range of embedded systems projects, from basic LED control and motor speed regulation to complex multi-actuator circuits, communication bus interfacing (I^2C, SPI, UART), and sensor integration (IMU, DHT, PIR, LDR, *etc.*). The selection criteria focused on emulating real-world embedded systems, robotics, and IoT hardware requests, ensuring a rigorous test of the system's ability to handle complexity, ambiguity, and technical

[4]Minor run-to-run variance may occur despite temperature 0 due to API-level nondeterminism.

constraints. The CircuitLM benchmark thus primarily targets the embedded systems domain. To systematically assess model limitations, the dataset is stratified into three distinct difficulty tiers—Easy, Medium, and Adversarial—ensuring a balanced distribution of complexity across the benchmark. To support external validity and future reproducibility, the complete dataset, including all prompts and canonical schemas, will be open-sourced upon publication.

D. Evaluation Methodology

To evaluate the physical viability of LLM-generated schematics, we implement a deterministic **ERC** framework inspired by standard Electronic Design Automation (EDA) workflows. Functioning as a safety-first verification layer, the system scans circuits for electrical violations and produces structured error reports categorized by severity.

Technical Implementation. The pipeline utilizes a topological graph analyzer that parses schematics into a bipartite graph of components, pins, and galvanic nets. Passive components are modeled using weighted traversal paths to mathematically distinguish direct shorts from resistive bridges. The system strictly enforces logic-level matching ($V_{\text{out}} > V_{\text{in_max}}$), detects floating inputs or net contention, and validates component-specific requirements (*e.g.,* PWM mapping) against a curated hardware **Knowledge Base (KB)**. This KB contains constraints for maximum operating voltages, current limits, pin multiplexing, and inductive load requirements.

Error Classification. Each model is tested three times. Detected issues are classified into four tiers:

- **Fatal (Safety-Critical):** Topological violations causing immediate damage, such as VCC–GND shorts identified via shortest-path traversal.
- **Major (Functional):** Faults guaranteeing system failure, including logic-level mismatches, missing current-limiting resistors, or absent flyback diodes.
- **Minor (Performance):** Suboptimal configurations, such as technically valid but poorly chosen passive values.
- **Warnings (Best Practices):** Deviations from engineering conventions, such as missing decoupling capacitors.

Primary Metric: Pass@1. Generative performance is measured using a binary **Pass@1** metric. A schematic is a "Pass" only if the ERC detects **zero Fatal and zero Major errors**. Minor errors and warnings are recorded for diagnostics but do not trigger failure. This criterion mirrors real-world standards where a single critical fault renders a board unusable. OOD halts are counted as automatic failures.

Blind Expert Verification. To validate the automated evaluation, three electrical engineering experts who underwent a calibration phase utilizing a strict, predefined error taxonomy, conducted a blind review of the circuits and ERC reports. Inter-rater reliability was quantified using **Fleiss' kappa** (κ) to provide a statistical measure of agreement among evaluators regarding circuit correctness and diagnostic accuracy. we extracted a stratified random sample comprising 25% of the generated circuits from each of the five frontier models, balanced across difficulty tiers for blind manual review.

Secondary Meta-Evaluation: LLM-as-a-Judge. ERC ensures electrical safety and connectivity but lacks the semantic context required to verify higher-level functionality in complex digital systems. ERC cannot guarantee the circuit works. It only checks electrical connection rules. Microcontrollers such as Arduino and ESP32 use highly multiplexed GPIOs and strict protocol requirements; thus, graph-based ERC can confirm connections and detect shorts but cannot verify higher-level logic—such as correct routing of SPI signals (MOSI/MISO), proper crossing of UART TX/RX lines, or avoiding address conflicts on I^2C buses. To address this limitation, we introduce a meta-evaluator following the *LLM-as-a-Judge* paradigm. A separate QA Agent evaluates protocol correctness, logical routing, and firmware plausibility using a different model (`Claude Sonnet 4.5`), excluded from the generator pool to avoid self-preference bias. Errors are classified into four fixed categories same as ERC but with more generalized instructions (*e.g.*, fatal errors are immediate circuit failure or damaged components, major errors mean rendering the circuit nonfunctional). The model also assumes *board-level reality*, recognizing that common modules may include onboard regulators and supporting circuitry.

V. Results

A. Physical Viability and Deterministic ERC Performance

The results in Table II demonstrate remarkably high reliability across all models. Gemini and Qwen led with Pass@1 rates of 88% and 87%, respectively. A key finding is the near-total elimination of fatal errors ($\mu = 0.0$ for nearly all backends), indicating that the CoT planning phase effectively forces models to identify VCC/GND rails and avoid critical short circuits. GPT was the only model exhibiting non-zero average fatal errors ($\mu = 0.0 \pm 0.2$), though still at an extremely low frequency.

B. Error Severity Analysis

While fatal errors were rare, major errors—primarily involving logic level mismatches or missing current-limiting resistors in multi-component systems—remained the primary failure mode. Models occasionally overlooked voltage domain boundaries (*e.g.,* 5V *vs.* 3.3V) when interfacing heterogeneous components. Deepseek showed a unique tendency for minor errors and warnings ($\mu = 0.3 \pm 0.9$ for warnings), often flagged by the ERC for missing decoupling capacitors on active components. Inter-rater reliability analysis demonstrates near-perfect consensus between the human experts and the ERC engine in classifying Fatal and Major violations, achieving a Fleiss' kappa of $\kappa = 0.82$ (95% CI $[0.74, 0.93]$). Agreement was near-perfect for safety-critical tiers (Fatal $\kappa = 0.94$, Major $\kappa = 0.88$), but slightly lower for subjective best-practices (Minor $\kappa = 0.73$, Warning $\kappa = 0.68$), ultimately confirming that CircuitLM reliably generates professionally viable, safe, and electrically correct hardware.

TABLE I
PERFORMANCE COMPARISON: FULL-CONTEXT ZERO-SHOT BASELINE VS. CIRCUITLM PIPELINE

Model	Zero-Shot Baseline				CircuitLM			
	ERC Pass@1 ↑	LLM Pass@1 ↑	LLM Fatal ↓	LLM Major ↓	ERC Pass@1 ↑	LLM Pass@1 ↑	LLM Fatal ↓	LLM Major ↓
Gemini 2.5 Flash	84%	51%	0.0 ± 0.3	1.2 ± 1.5	88% (+4%)	53% (+2%)	0.0 ± 0.2	1.1 ± 1.3
Qwen-3 235B	79%	33%	0.9 ± 0.4	1.7 ± 1.4	87% (+8%)	38% (+5%)	0.4 ± 0.7	1.8 ± 1.4
Deepseek v3.1	77%	39%	0.5 ± 0.7	1.7 ± 1.3	86% (+9%)	40% (+1%)	0.3 ± 0.8	1.5 ± 1.6
GPT-5 mini	85%	21%	0.1 ± 0.4	1.8 ± 1.2	83% (-2%)	23% (+2%)	0.1 ± 0.2	1.3 ± 1.4

TABLE II
ERC EVALUATION RESULTS: PASS@1 RATES AND ERRORS PER PROMPT ($\mu \pm \sigma$)

Model	Pass@1	Fatal	Major	Minor	Warning
Gemini	**88%**	0.0 ± 0.0	0.1 ± 0.5	0.0 ± 0.0	0.0 ± 0.0
Qwen	87%	0.0 ± 0.0	0.2 ± 0.6	0.0 ± 0.0	0.0 ± 0.0
Deepseek	86%	0.0 ± 0.0	0.2 ± 0.5	0.1 ± 0.8	0.3 ± 0.9
GPT	83%	0.1 ± 0.2	0.3 ± 0.7	0.0 ± 0.0	0.0 ± 0.0
Llama	84%	0.1 ± 0.2	0.2 ± 0.6	0.0 ± 0.0	0.0 ± 0.0

C. LLM-as-a-Judge Meta-Evaluation

A significant "evaluation gap" is observed between the deterministic ERC Pass@1 (Table II) and the LLM-as-Judge Pass@1 (Table III). While Gemini achieved an 88% ERC pass rate, its LLM-as-Judge pass rate dropped to 53%. Similar trends are visible across all backends, with Llama falling from 84% to 21%. This discrepancy highlights the value of semantic validation in circuit generation. Blind expert audits of the LLM judge's outputs yielded a substantial Fleiss' kappa of $\kappa = 0.78$ (95% CI $[0.65, 0.88]$), confirming the agent is not hallucinating faults, but reliably validating complex, protocol-driven hardware design. The LLM-as-Judge demonstrated superior context awareness and vast internal electrical knowledge, identifying sophisticated design flaws that bypassed the deterministic checker's static rules. Table V shows the breakdown of Pass@1 scores across prompt difficulty levels for both ERC and LLM-based evaluations.

TABLE III
LLM-AS-JUDGE EVALUATION RESULTS: PASS@1 RATES AND ERRORS PER PROMPT ($\mu \pm \sigma$)

Model	Pass@1	Fatal	Major	Minor	Warning
Gemini	53%	0.0 ± 0.2	1.1 ± 1.3	0.5 ± 0.7	2.5 ± 1.2
Deepseek	40%	0.1 ± 0.4	1.3 ± 1.4	0.5 ± 0.7	2.7 ± 1.5
Qwen	38%	0.3 ± 0.8	1.5 ± 1.6	0.5 ± 0.7	2.5 ± 1.2
GPT	23%	0.4 ± 0.7	1.8 ± 1.4	0.7 ± 0.7	3.3 ± 1.0
Llama	21%	0.4 ± 0.8	2.1 ± 1.6	0.6 ± 0.8	2.4 ± 1.0

The LLM judge frequently caught errors in digital interfaces that the ERC engine, which primarily checks for net continuity and logic levels, overlooked. This includes UART RX/TX confusion (*e.g.,* connecting a microcontroller's TX pin directly to another device's TX pin) and SPI protocol miswiring or omitting required pull-up resistors on bus lines. Beyond standard GPIO, the LLM judge identified mismatches between a pin's physical capabilities and its assigned function. A recurring example was the use of *non-PWM-capable pins* for pulse-width modulation tasks or the selection of multiplexed pins that were already reserved for an active SPI/UART bus, avoiding resource contention that rule-based checks might miss. While the ERC flags direct logic-level mismatches, the LLM judge identified "brownout" risks and *undervoltage conditions* that are technically within logic tolerances but functionally invalid. For instance, attempting to drive a 5V-required inductive load (*e.g.,* a 5V relay or buzzer) with a 3.3V GPIO signal was flagged as a major error, even if the ERC registry categorized the connection as a valid digital interface. Figure 8 illustrates the errors identified by the LLM in a sample circuit.

VI. ABLATION STUDY: ZEROSHOT BASELINE

To evaluate the necessity of the hybrid retrieval module and the multi-agent orchestration within the CircuitLM architecture, we conduct a baseline ablation study on the top 3 models.

A. Ablation Setup

A standard zero-shot model must either generate circuit descriptions without grounding—leading to systematic component hallucination and a failure to produce visually renderable schematics—or receive the entire component library in context. To establish the strongest possible baseline, we bypassed CircuitLM's retrieval agent and CoT agent entirely. Instead, the full, unfiltered component library, complete with all semantic descriptions and exact pin definitions, was injected directly into a single system prompt. Additionally, we run the ablation removing only the CoT agent. This configuration forced the generator LLM to select components and route connections in a single pass. To account for stochastic variation in model outputs, we ran each experiment 3 times for every model and report the average performance along with the variance.

B. Ablation Results

The two approaches diverge significantly in terms of computational overhead and architectural scalability. The zero-shot baseline consumes ∼15,000 tokens per request due to the exhaustive library injection. In contrast, the CircuitLM pipeline's targeted retrieval and structured planning use only ∼3,000 tokens per request. While the zero-shot baseline exhibited lower average latency (∼3 seconds *vs.* ∼8 seconds for CircuitLM due to multiple API calls), the full-context approach fundamentally fails to scale. As component libraries expand toward industrial proportions, injecting thousands of components data into a single prompt becomes cost-prohibitive and inevitably breaches the context window limits of most

models. Notable in our testing was the exclusion of **Llama-3.3**, which consistently failed to generate outputs in the zero-shot setting due to immediate context overflows.

Our quantitative analysis in Table I indicates that for high-parameter "reasoning" models, the performance gap is relatively small. This is largely because modern frontier models are inherently trained to perform internal Chain-of-Thought (CoT) reasoning. However, incorporating an explicit CircuitLM CoT planning phase adds a crucial safety layer, helping ensure that complex circuits are constructed correctly, as demonstrated in Table IV.

Interestingly, we observed a performance degradation in **GPT-5-mini**. We hypothesize that for distilled, highly optimized models, overly prescriptive planning instructions may interfere with internal heuristic efficiencies. Nevertheless, for the majority of the backends, the CircuitLM pipeline provides the robust, context-efficient architecture necessary for reliable Electronic Design Automation (EDA).

VII. Limitations and Conclusion

A notable limitation of our evaluation framework is the absence of simulator-in-the-loop verification (*e.g.,* automated Wokwi or Proteus co-simulation). Limitations also include computational overhead from iterative multi-agent queries and variable latency depending on model and API traffic. LLM-based validation relied on a single evaluator. Our evaluation focused primarily on digital and embedded electronics, and we did not experimentally assess performance on analog circuits (*e.g.*, op-amp–based designs). Additionally, converting `CircuitJSON` into other representations, such as a netlist, would require robust scripting and additional engineering effort, which falls outside the scope of this work.

CircuitLM introduces a multi-agent framework that translates natural-language intent into physically realizable electronic schematics. By incorporating a CoT phase and an evaluation framework (deterministic ERC and LLM-as-a-Judge) we significantly improved design reliability across multiple frontier models. Future work will focus on a heterogeneous multi-LLM evaluation system, utilizing agreement-based consensus and iterative feedback loops to validate and correct connectivity logic before synthesis. We will explore a direct transpilation layer from CircuitLM's connection arrays into Python-based SKiDL definitions, allowing the automated generation of KiCad-compatible netlists. We also plan to explore ensemble architectures to improve scalability across complex industrial designs. Finally, we propose the `CircuitJSON` format as a standardized bridge between natural language and hardware description, encouraging its adoption for future dataset creation in the EDA community.

References

[1] P. Jansen, "From words to wires: Generating functioning electronic devices from natural language descriptions," 2023. [Online]. Available: https://arxiv.org/abs/2305.14874

[2] S. Talebzadeh *et al.*, "Machine learning in eda: A survey," *ACM Transactions on Design Automation of Electronic Systems*, 2021.

[3] S. Thakur *et al.*, "Verilogeval: Evaluating large language models for verilog code generation," *arXiv preprint arXiv:2303.05461*, 2023.

[4] J. Blocklove *et al.*, "Chip-chat: Challenges and opportunities in conversational hardware design," in *Proceedings of the 2023 ACM/IEEE Workshop on Machine Learning for CAD*, 2023.

[5] R. Matsuo, S. Uhlich, A. Venkitaraman, A. Bonetti, C.-Y. Hsieh, A. Momeni, L. Mauch, A. Capone, E. Ohbuchi, and L. Servadei, "Schemato – an llm for netlist-to-schematic conversion," 2025. [Online]. Available: https://arxiv.org/abs/2411.13899

[6] C. Liu and D. Chitnis, "Eeschematic: Multimodal-llm based ai agent for schematic generation of analog circuit," 2025. [Online]. Available: https://arxiv.org/abs/2510.17002

[7] K. Ye, Q. Yang, Z. Lu, H. Yu, T. Cui, R. Bai, and L. Shen, "Llm4netlist: Llm-enabled step-based netlist generation from natural language description," *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, 2025.

[8] Z. Wan, Y. LI, X. Wen, Y. Song, H. Wang, L. Yang, M. Schmidt, J. Wang, W. Zhang, S. Hu, and Y. Wen, "ReMA: Learning to meta-think for LLMs with multi-agent reinforcement learning," in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. [Online]. Available: https://openreview.net/forum?id=ur295YVtmt

[9] O. Siddique, J. Alam, M. J. R. Rafy, S. R. Raiyan, H. Mahmud, and M. K. Hasan, "Physicseval: Inference-time techniques to improve the reasoning proficiency of large language models on physics problems," *arXiv preprint arXiv:2508.00079*, 2025.

[10] S. Hong *et al.*, "Metagpt: Meta programming for multi-agent collaborative framework," *arXiv preprint arXiv:2308.00352*, 2024.

[11] P.-H. Chen, Y.-S. Lin, W.-C. Lee, T.-Y. Leu, P.-H. Hsu, A. Dissanayake, S. Oh, and C.-S. Chiu, "Menter: A fully-automated multi-agent workflow for end-to-end rf/analog circuits netlist design," 2025. [Online]. Available: https://arxiv.org/abs/2505.22990

[12] J. Pan, G. Zhou, C.-C. Chang, I. Jacobson, J. Hu, and Y. Chen, "A survey of research in large language models for electronic design automation," 2025. [Online]. Available: https://arxiv.org/abs/2501.09655

[13] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," 2021. [Online]. Available: https://arxiv.org/abs/2005.11401

[14] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-

thought prompting elicits reasoning in large language models," 2023. [Online]. Available: https://arxiv.org/abs/2201.11903

[15] D. Zhou, N. Schärli, L. Hou, J. Wei, N. Scales, X. Wang, D. Schuurmans, C. Cui, O. Bousquet, Q. Le, and E. Chi, "Least-to-most prompting enables complex reasoning in large language models," 2023. [Online]. Available: https://arxiv.org/abs/2205.10625

[16] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," 2023. [Online]. Available: https://arxiv.org/abs/2203.11171

[17] K. Asparuhova, S. Daniela, S. Asenov, H. Kanevski, and A. Parushev, "Using wokwi simulator to support engineering student learning in microcontrollers and sensors," 09 2024, pp. 1–4.

[18] M. Schönfeld and J. Pfeffer, *Fruchterman/Reingold (1991): Graph Drawing by Force-Directed Placement*. Wiesbaden: Springer Fachmedien Wiesbaden, 2019, pp. 217–220. [Online]. Available: https://doi.org/10.1007/978-3-658-21742-6_49

[19] C. Y. Lee, "An algorithm for path connections and its applications," *IRE Trans. Electron. Comput.*, vol. 10, pp. 346–365, 1961. [Online]. Available: https://api.semanticscholar.org/CorpusID:40700386

[20] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating large language models trained on code," 2021. [Online]. Available: https://arxiv.org/abs/2107.03374

[21] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, "Program synthesis with large language models," 2021. [Online]. Available: https://arxiv.org/abs/2108.07732

[22] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv *et al.*, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.

[23] Y. Zhang, M. Li, D. Long, X. Zhang, H. Lin, B. Yang, P. Xie, A. Yang, D. Liu, J. Lin, F. Huang, and J. Zhou, "Qwen3 embedding: Advancing text embedding and reranking through foundation models," 2025. [Online]. Available: https://arxiv.org/abs/2506.05176

[24] A. Singh, A. Fry, A. Perelman, A. Tart, A. Ganesh, A. El-Kishky, A. McLaughlin, A. Low, A. Ostrow, A. Ananthram *et al.*, "Openai gpt-5 system card," *arXiv preprint arXiv:2601.03267*, 2025.

[25] G. Team, R. Anil, S. Borgeaud, J.-B. Alayrac, J. Yu, R. Soricut, J. Schalkwyk, A. M. Dai, A. Hauth, K. Millican *et al.*, "Gemini: a family of highly capable multimodal models," *arXiv preprint arXiv:2312.11805*, 2023.

[26] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan *et al.*, "Deepseek-v3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[27] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

APPENDIX

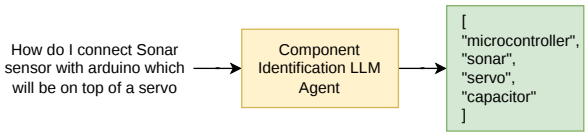


Fig. 3. Input output data of component identification agent

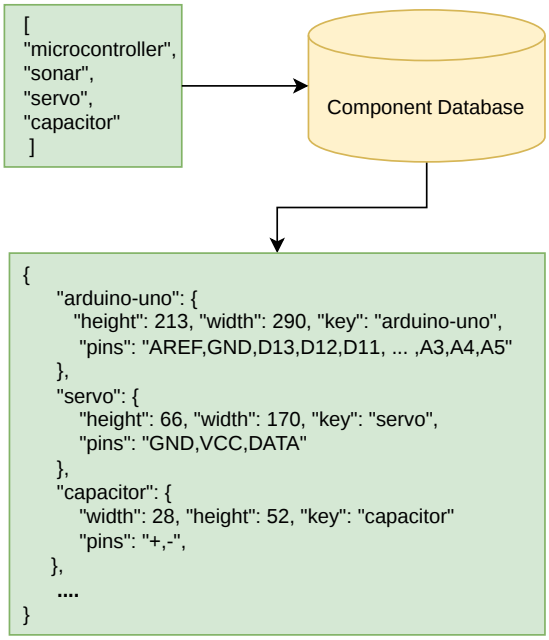


Fig. 4. Components pin out information is retrieved by using the component's name from the database

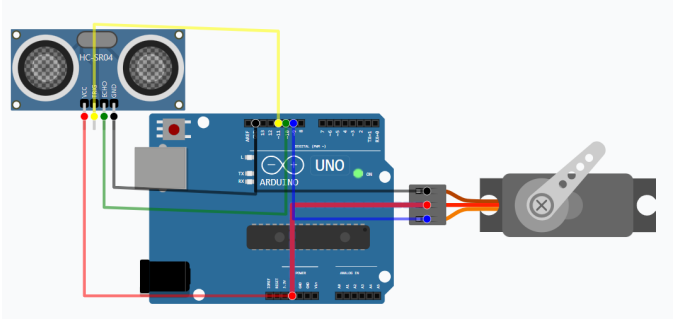


Fig. 5. Circuit Schematic of a simple radar system generated by the visualization engine

TABLE IV
PERFORMANCE OF LLM-AS-JUDGE ACROSS PROMPT DIFFICULTY LEVELS (PERCENTAGE) AFTER REMOVING THE CHAIN-OF-THOUGHT AGENT

Model	Easy (%)	Medium (%)	Hard (%)	Overall (%)
Gemini 2.5 Flash	72.46	58.00	20.00	51.65
Deepseek v3.1	55.00	37.00	29.00	38.83
Qwen-3 235B	63.45	52.66	17.00	36.86
GPT-5 mini	44.00	32.00	14.00	26.00
Llama-3.3 70B	44.26	36.33	2.00	21.00

Note: Pass@1 scores are averaged across all runs. For each difficulty level in each run, the score is calculated as $100 \times (\text{Passed} \div \text{Total})$, where Total is the number of prompts at that difficulty.

TABLE V
PERFORMANCE BREAKDOWN BY PROMPT DIFFICULTY (PERCENTAGE) FOR BOTH ERC AND LLM-BASED EVALUATION

Model	Easy (%)		Medium (%)		Hard (%)	
	ERC	LLM	ERC	LLM	ERC	LLM
Gemini 2.5 Flash	90.00	75.00	80.00	60.00	92.00	40.00
Deepseek v3.1	95.00	55.00	80.00	36.67	86.00	36.00
Qwen-3 235B	90.00	65.00	80.00	53.33	90.00	18.00
GPT-5 mini	85.00	45.00	80.00	30.00	84.00	10.00
Llama-3.3 70B	85.00	45.00	76.67	36.67	88.00	2.00

Note: Pass@1 scores are averaged across all runs. For each difficulty level in each run, the score is calculated as $100 \times (\text{Passed} \div \text{Total})$, where Total is the number of prompts at that difficulty.

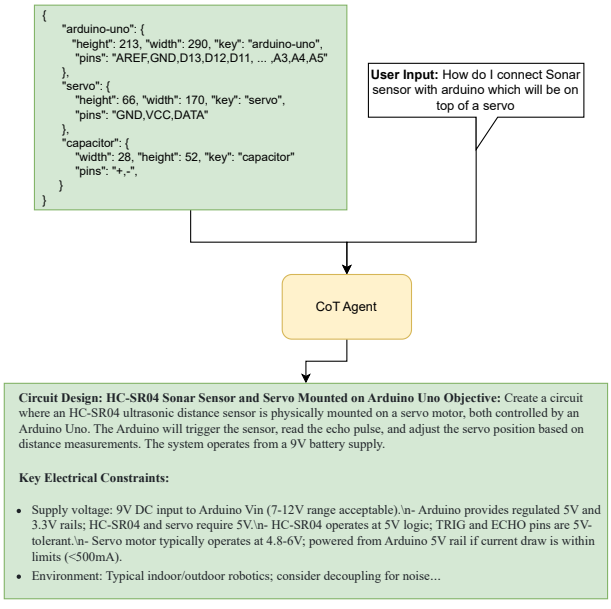


Fig. 6. Workflow of the chain of thought agent, it takes the user's prompt and the retrieved components list and generates a reasoning for building the circuit

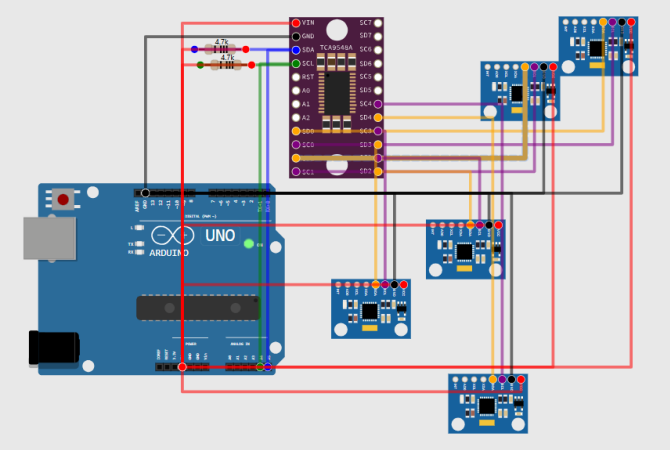


Fig. 7. Circuit Schematic of Sign language glove using 5 IMU sensors for 5 fingertips generated by the visualization engine

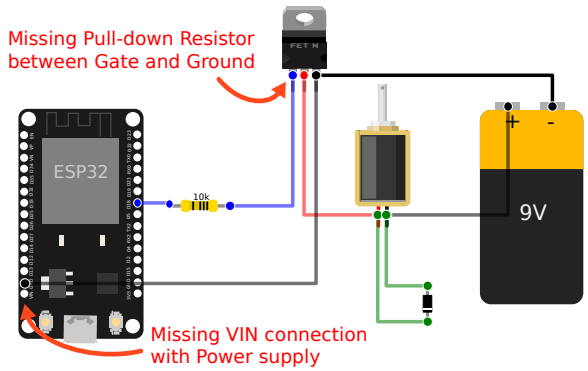


Fig. 8. LLM-as-Judge evaluation of a CircuitLM-generated schematic for a solenoid driver. For the prompt "How do I wire an N-channel MOSFET to drive a solenoid using PWM signal?", the framework generates a structurally compliant baseline using an ESP32 and an N-channel FET. However, the QA Agent correctly flags practical electrical faults, specifically identifying a missing pull-down resistor between the gate and ground and a missing VIN connection with the power supply.

```
CircuitJSON
  version : Number
  author  : String
  parts : List of Part
  connections : List of Connection

Part
  type  : String   // e.g. "arduino-uno"
  id    : String   // unique instance ID
  top   : Number   // Y-coordinate
  left  : Number   // X-coordinate
  attrs : Map<String, Any>
  rotate: Number (optional)

Connection
  startPin: String  // e.g. "arduino:5V"
  endPin  : String  // e.g. "l298n:5V"
  color   : String  // wire color
  route   : List<String>  // routing instructions
```

Listing 1. `CircuitJSON` Schema Definition. The top, left, rotate are for placement of components, attrs is a flexible map for additional information, and parametric values.

TABLE VI
QUALITATIVE COMPARISON OF CIRCUITLM WITH RECENT LLM-BASED HARDWARE GENERATION FRAMEWORKS

Framework	**Target Domain**	**Input Modality**	**Output Format**	**Component Retrieval**	**Automated Rule Checking**	**Visual Rendering**
VerilogEval [3]	Digital RTL / Logic	Natural Language	Verilog	✗	Syntax / Compilation	✗
LLM4Netlist [7]	Digital Logic Gates	Natural Language	SPICE Netlist	✗	Simulation-based	✗
Schemato [5]	Analog Circuits	Netlist	Schematic	N/A	✗	✓
EESchematic [6]	Analog Circuits	Multimodal (Text/Image)	Schematic	✗	✗	✓
MenTeR [11]	RF / Analog	Natural Language	Netlist / Layout	✓ (Diagram-aware)	Simulation-based	✗
CircuitLM (Ours)	**Embedded Systems**	**Natural Language**	**CircuitJSON**	**✓ (Components dataset)**	**✓ (ERC & LLM)**	**✓ (Interactive)**